

NAME - ARFHA.M.RANEENNUR

USN - 2VX24UE012

EXPT.NO - 9

TITLE - Minimum Cost Spanning Tree using PRIM'S ALGORITHM

```
In [2]: 1 # Taking input from the user
2 n = int(input("Enter number of vertices in the graph: "))
3
4 print("Enter the adjacency matrix where each row is separated by spaces:")
5 graph = []
6 for i in range(n):
7     row = input().split()
8     graph.append([int(x) for x in row])
9
10 # Function to get the vertex with minimum key value
11 def get_min_key_vertex(key, in_mst):
12     return min(
13         (v for v in range(len(key)) if not in_mst[v]),
14         key=lambda v: key[v],
15         default=-1
16     )
17
18 # Prim's Algorithm function
19 def Prim_MCST(graph):
20     vertices = len(graph)
21
22     parent = [-1] * vertices
23     key = [float('inf')] * vertices
24     in_mst = [False] * vertices
25
26     key[0] = 0
27     total_cost = 0
28
29     for _ in range(vertices):
30         u = min(
31             (v for v in range(vertices) if not in_mst[v]),
32             key=lambda v: key[v],
33             default=-1
34         )
35
36         if u == -1:
37             break
38
39         in_mst[u] = True
40
41         for v in range(vertices):
42             if graph[u][v] != 0 and not in_mst[v] and graph[u][v] < key[v]:
43                 key[v] = graph[u][v]
44                 parent[v] = u
45
46     total_cost = sum(key[1:]) # Sum of weights in MST
47     return total_cost
48
49 # Function call
50 print("=====")
51 min_cost = Prim_MCST(graph)
52 print("Minimum Cost of Spanning Tree using Prim's Algorithm:", min_cost)
53 print("=====")
```

```
Enter number of vertices in the graph: 6
Enter the adjacency matrix where each row is separated by spaces:
0 4 6 0 0 0
4 0 0 0 0 0
6 0 0 0 5 0
0 0 0 0 2 1
0 0 5 2 0 0
0 0 0 1 0 0
=====
Minimum Cost of Spanning Tree using Prim's Algorithm: 18
=====
```